

Compiler Design — Lexical & Syntax Analysis + Ambiguous Grammar

1. Lexical Analyzer

Lexical Analyzer is the first phase of a compiler. It reads the source program **character by character** and converts **lexemes into tokens**.

Main Functions

1. **Lexeme → Token conversion**
2. Removes **white spaces**
3. Removes **comments**
4. Detects lexical errors and keeps track of **line numbers**.

Example

```
int max(x,y)
{
    return (x > y ? x : y);
}
```

Here:

- int → keyword/token
- max → identifier/token
- (,), {, }, ,, ;, > → operators/delimiters/tokens
- Comments and unnecessary spaces are **not counted as tokens**.

⚠ **GATE point:** A complete string literal is treated as **one token**, not as separate tokens character-by-character.

2. Syntax Analyzer / Parser

The **Syntax Analyzer (Parser)** takes the tokens produced by lexical analysis and checks whether they follow the given **grammar**.

The basic flow is:

Source Program → Lexical Analyzer → Tokens → Syntax Analyzer → Parse Tree

The grammar is represented as:

$$G = (V, T, P, S)$$

where:

Symbol Meaning

V	Set of variables / non-terminals
T	Set of terminals
P	Set of productions/rules
S	Start symbol

For example:

$$E \rightarrow E + E$$

$$E \rightarrow E * E$$

$$E \rightarrow id$$

Here E is a variable, while $+$, $*$, and id are terminals.

3. Derivation

A **derivation** means generating a string from the start symbol by repeatedly applying production rules.

For:

$$E \rightarrow E + E \mid E * E \mid id$$

To generate:

$$id + id * id$$

we start from:

$$E$$

and repeatedly replace a variable using a production.

Leftmost Derivation (LMD)

At every step, replace the **leftmost non-terminal** first.

Example:

$$E \Rightarrow E + E$$

$$\Rightarrow id + E$$

$$\Rightarrow id + E * E$$

$$\Rightarrow id + id * E$$

$$\Rightarrow id + id * id$$

Rightmost Derivation (RMD)

At every step, replace the **rightmost non-terminal** first.

GATE shortcut:

LMD $\rightarrow$ expand **leftmost** variable

RMD $\rightarrow$ expand **rightmost** variable

The same string can have different derivations.

4. Parse Tree

A **parse tree** represents the syntactic structure of a string according to a grammar.

Important points:

- Root = **Start symbol**
- Internal nodes = **Variables/non-terminals**
- Leaf nodes = **Terminals**
- Reading the terminals **left $\rightarrow$ right** gives the **yield** of the parse tree.

So if the leaves are:

$$id + id * id$$

the yield is:

$$id + id * id$$

The lecture emphasizes that the yield is obtained by traversing the tree **top-down, left-to-right**, writing the terminals encountered.

5. Ambiguous Grammar

A grammar is **ambiguous** if there exists at least one string generated by the grammar that has **more than one distinct parse tree**.

Most important GATE condition:

Same string + 2 different Parse Trees = Ambiguous Grammar

Also, if the same string has:

- more than one **LMD**, or
- more than one **RMD**,

then the grammar is ambiguous.

Example

Consider:

$$E \rightarrow E + E \mid E * E \mid id$$

For:

$$id + id * id$$

two interpretations are possible:

$$(id + id) * id$$

and

$$id + (id * id)$$

Thus, two different parse trees are possible.

Therefore:

Grammar is ambiguous

This is why expression grammars need **operator precedence and associativity** to remove ambiguity.

6. How to Prove a Grammar is Ambiguous?

When asked in GATE:

Step 1

Choose a suitable string generated by the grammar.

Step 2

Construct **Parse Tree 1**.

Step 3

Construct **Parse Tree 2** for the **same string**.

Step 4

Check that both trees have the same yield.

Step 5

Conclude:

Since the same string has two distinct parse trees, the grammar is ambiguous.

Important: You only need **one counterexample string** to prove ambiguity. You do **not** need to show that every string is ambiguous.

7. Ambiguity & Undecidability — GATE Point ★

The lecture notes that there is **no general algorithm** that can decide ambiguity for every context-free grammar; the ambiguity problem is **undecidable**.

So in typical problems, we try to **find a suitable string** and construct multiple parse trees.



GATE Quick Revision

Concept

Lexical Analyzer

Remember

Lexemes → Tokens

Lexical Analyzer	Removes comments & whitespace
Syntax Analyzer	Checks token sequence using grammar
Grammar	$G = (V, T, P, S)$
V	Variables / Non-terminals
T	Terminals
P	Productions
S	Start symbol
LMD	Leftmost variable first
RMD	Rightmost variable first
Parse Tree	Shows syntactic structure
Yield	Terminals read left → right
Ambiguous Grammar	≥ 2 parse trees for same string
To prove ambiguity	Find one such string
General ambiguity problem	Undecidable

One-line memory trick

Lexer → Tokens → Parser → Grammar → Parse Tree → Ambiguity check

This is the core chain you should keep in mind for GATE.

Ambiguity & Disambiguation

1. Ambiguous Grammar ★ ★ ★

A grammar is **ambiguous** if **at least one string** generated by it has **more than one parse tree**.

1 String + 2 Parse Trees $\Rightarrow$ Ambiguous

It can also be detected through:

- more than one **LMD**, or
- more than one **RMD**.

Important: Ambiguity prove karne ke liye **sirf one string** with two parse trees is enough.

2. Why does ambiguity occur?

Expression grammar mein mainly 2 reasons:

A. Associativity problem

Example:

$id + id + id$

Possible:

$(id + id) + id$

or

$id + (id + id)$

If + is supposed to be **left associative**, first structure chahiye.

Lecture explains that unrestricted grammar allows tree growth in both directions, causing multiple parse trees.

B. Precedence problem

Example:

$id + id * id$

Normally:

$* > +$

So desired structure:

$id + (id * id)$

If grammar does not specify precedence, another parse tree may give:

$(id + id) * id$

Thus ambiguity.

3. Disambiguation ★

Disambiguation = ambiguous grammar ko unambiguous grammar mein convert karna.

Lecture calls these **disambiguating rules**.

Mainly 2 rules use karo:

Associativity → Recursion

Precedence → Different Levels

4. Left Recursion → Left Associativity ★ ★ ★

Left recursive grammar

A production is directly left recursive if:

$A \rightarrow A\alpha$

Example:

$E \rightarrow E + id \mid id$

Here RHS ka **leftmost symbol** E = LHS E.

Therefore it is **left recursive**.

This forces the expression to grow toward the left and gives **left associativity**.

Example

$id + id + id$

becomes:

$(id + id) + id$

So:

$Left\ Recursion \Rightarrow Left\ Associativity$

5. Right Recursion → Right Associativity

For a right-associative operator like exponentiation:

2^{3^2}

desired interpretation:

$2^{(3^2)}$

So use right recursion:

$F \rightarrow id^F \mid id$

Therefore:

Right Recursion $\Rightarrow$ Right Associativity

Lecture specifically uses exponentiation as the right-associative example.

6. Precedence $\rightarrow$ Different Grammar Levels

Suppose:

$*$ $>$ $+$

Use:

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow id$

Think:

$E \rightarrow +$ level

$\downarrow$

$T \rightarrow *$ level

$\downarrow$

$F \rightarrow id$

This prevents $*$ and $+$ from getting mixed arbitrarily.

For:

$id + id * id$

grammar forces:

$id + (id * id)$

because once we move from E to T , T can generate $*$ but cannot go back and generate $+$.

7. Very Important Rule

For associativity:

Recursion direction decides associativity.

Recursion	Associativity
-----------	---------------

Left recursion	Left associative
----------------	------------------

Right recursion	Right associative
-----------------	-------------------

For precedence:

Grammar levels decide precedence.

If:

$A \rightarrow \dots B \dots$

and B is at a deeper/lower level, its operator gets **higher precedence** according to the lecture's level convention.

8. Standard Unambiguous Expression Grammar

For:

$+, *, \wedge$

where:

$\square > * > +$

and $+$, $*$ are left associative while $\wedge$ is right associative:

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow P^F \mid P$$

$$P \rightarrow id$$

Notice:

- $E \rightarrow E+T \rightarrow$ **left recursion** $\rightarrow +$ left associative
- $T \rightarrow T*F \rightarrow$ **left recursion** $\rightarrow *$ left associative
- $F \rightarrow P^F \rightarrow$ **right recursion** $\rightarrow ^$ right associative
- Different levels $\rightarrow$ **precedence**

Hence grammar is **unambiguous**.

9. Removing Left Recursion — Important Distinction



Yahan ek important confusion mat karna:

Left recursion ko remove karna $\neq$ ambiguity remove karna

For example:

$$E \rightarrow E + T \mid T$$

is left recursive and represents **left associativity**.

In **parser design**, especially top-down/recursive-descent parsing, left recursion may need to be eliminated.

Standard transformation:

$$A \rightarrow A\alpha \mid \beta$$

becomes:

$$A \rightarrow \beta A'$$

$$A' \rightarrow \alpha A' \mid \epsilon$$

But this transformation changes the **form of the grammar**, so don't automatically think:

“Left recursion = wrong.”

In this lecture, left recursion is intentionally used to **encode left associativity**.

10. Non-determinism — where it fits

Tumne **non-deterministic** bola, so GATE ke liye isko alag concept samjho:

Ambiguous grammar

Same string ke **multiple parse trees**.

Non-deterministic parsing situation

Parser ke paas kisi point par **multiple possible choices** ho sakte hain.

Ambiguity can cause such parsing choices, because parser doesn't have a unique parse structure to follow. Lecture notes that with an ambiguous grammar, the parser may not know which parse tree to use.

But: Don't write “**non-deterministic = ambiguous**”. They are related concepts, but **not the same definition**.



GATE Cheat Sheet

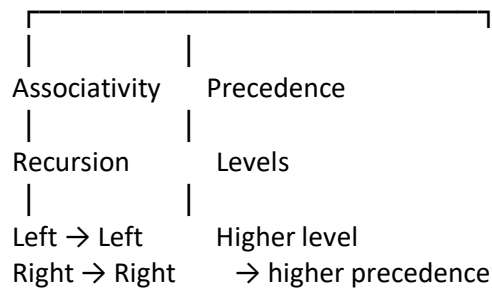
AMBIGUITY



Same string → 2 Parse Trees



DISAMBIGUATION



Must Remember:

1. Same string + ≥ 2 parse trees → **Ambiguous**
2. Left recursion → **Left associativity**
3. Right recursion → **Right associativity**
4. Different grammar levels → **Precedence**
5. * generally higher precedence than +
6. ^ is commonly right associative
7. Ambiguous grammar can sometimes be converted into an **unambiguous grammar** using disambiguating rules.
8. **Removing left recursion** is a separate parser-related transformation; don't confuse it with disambiguation.
9. Ambiguity problem for CFGs is **undecidable**; in these lecture examples, we use a suitable string and look for multiple parse trees.

Exam ke liye is lecture ka core actually ye hai:

Ambiguity → Associativity → Left/Right Recursion → Precedence → Disambiguation
--

That's the part you should focus on most.